وثيقة المشرف

Secure Code Analyzer: منصة ذكية لاكتشاف ثغرات الكود المصدري وتقييم الأمان

Intelligent Source Code Vulnerability Detection and Security Assessment Platform

نوع العمل	مشروع تخرج: منصة برمجية تعمل محلياً، مع واجهة ويب وسطر أوامر
المستودع	https://github.com/blackangiliq/graduate_project
الإصدار	1.2.0
لغة التنفيذ	Python 3.10 فأحدث
حجم الكود	نحو 12,150 سطر بايثون: 8,030 للنواة، 680 للواجهة، 2,050 للاختبارات، والباقي للأدوات والتقييم
الاختبارات	271 اختباراً آلياً، تعمل على لينكس وويندوز عبر GitHub Actions
القواعد	398 قاعدة كشف، منها 351 فعّالة
اللغات	تتعرّف المنصة على 22 لغة برمجة (54 امتداداً)
لغة الواجهة	العربية والإنجليزية

هذه الوثيقة تلخّص للمشرف ما يغطيه النظام، وكيف بُني، وما قيس فعلاً، وما لم يُختبر بعد. كل رقم فيها ناتج عن تشغيل فعلي مسجَّل في المستودع، وكل حدّ معروف مذكور في القسم 11.

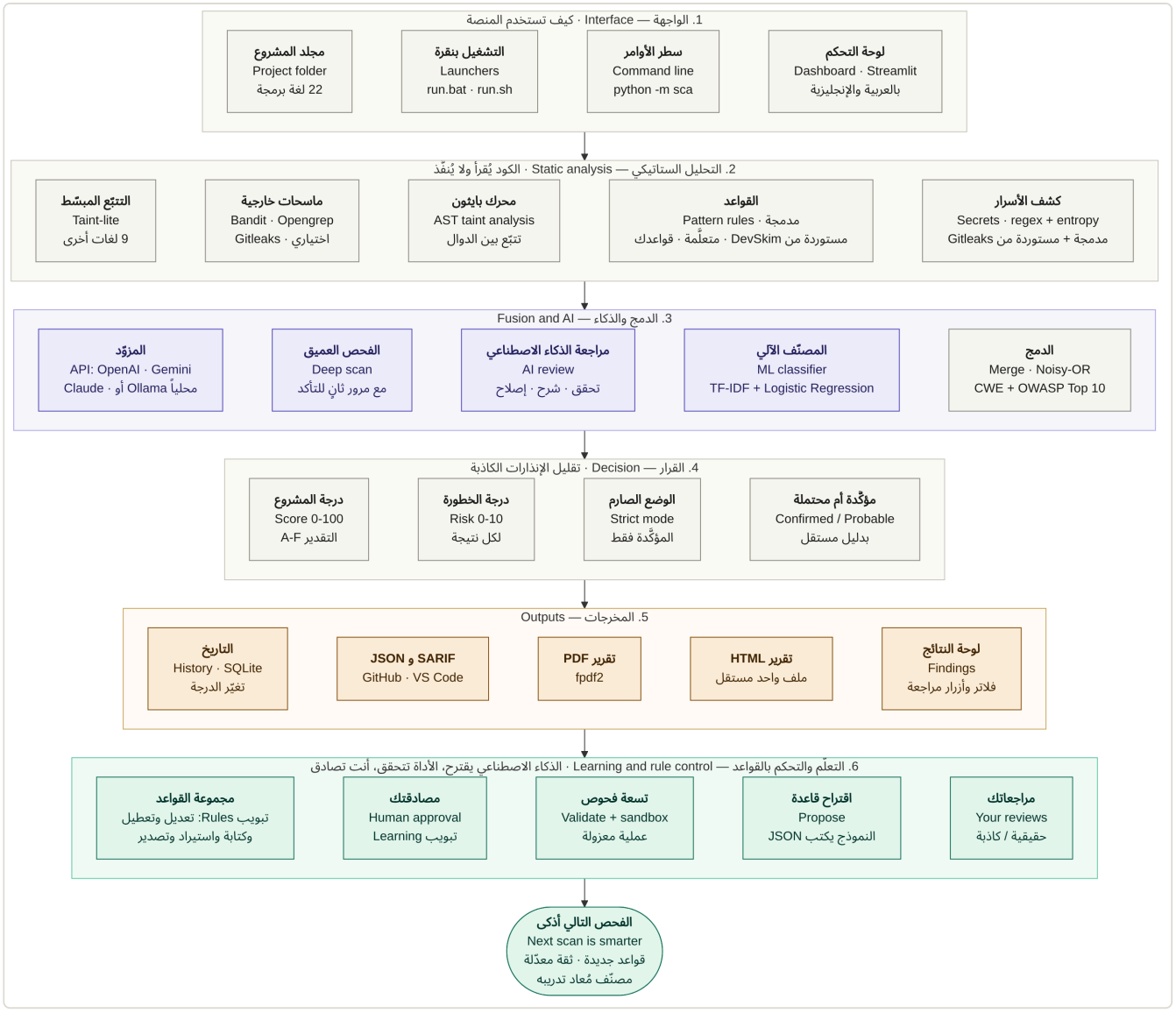
---

1. الملخص

المنصة تأخذ مجلد مشروع برمجي وتفحص كوده دون تنفيذه. تعمل فيها عدة محركات تحليل ستاتيكي معاً، ثم تُدمج بلاغاتها، ثم يراجعها مصنّف تعلّم آلة ونموذج لغوي (عبر API سحابي أو نموذج محلي)، ثم تُقسم النتائج إلى مؤكَّدة ومحتملة ويُحسب للمشروع درجة أمان من 100.

ما يميّز المشروع عن تشغيل أداة فحص جاهزة ثلاثة أمور:

1. **التعلّم بمصادقة بشرية.** ما يكتشفه الذكاء الاصطناعي يتحوّل إلى قاعدة مقترحة، تمرّ على تسعة فحوص آلية، ولا تُفعَّل إلا بعد موافقة المستخدم. بعدها تكشف الأداة الثغرة نفسها ستاتيكياً دون الحاجة إلى الذكاء الاصطناعي.
2. **التحكم الكامل بالقواعد.** كل قاعدة ظاهرة للمستخدم، وقابلة للقراءة والتعديل والتعطيل والاسترجاع، ويمكن كتابة قواعد جديدة وتجربتها من الواجهة.
3. **تقليل الإنذارات الكاذبة بدليل مستقل.** لا تُعتبر النتيجة مؤكَّدة لمجرد أن قاعدة طابقت سطراً، بل تحتاج دليلاً: تتبّع مدخلات المستخدم حتى السطر، أو تأكيد المراجعة، أو تأكيد المستخدم.



2. مشكلة البحث والأهداف

المشكلة. أدوات التحليل الستاتيكي المعروفة تعاني من أمرين: كثرة الإنذارات الكاذبة التي تجعل المطوّر يتجاهل التقرير، وثبات قواعدها فلا تتعلّم من المشروع الذي تفحصه. والنماذج اللغوية تكتشف ثغرات لا تصفها القواعد، لكنها بطيئة ومكلفة وقد تخطئ، ولا يصحّ الاعتماد على حكمها وحده.

الهدف. بناء منصة تجمع الطريقتين بحيث تعوّض كل واحدة ضعف الأخرى: القواعد سريعة ومجانية وقابلة للتدقيق، والنموذج اللغوي يكتشف الجديد ويشرح، وحلقة التعلّم تنقل اكتشافات النموذج إلى قواعد ثابتة بعد فحصها والمصادقة عليها.

الأهداف التفصيلية:

- فحص محلي لمجلد كامل بعدة لغات برمجة.
- تحليل أعمق من مطابقة الأنماط للغة واحدة على الأقل (بايثون: تتبّع البيانات بين الدوال).
- مراجعة النتائج وشرحها واقتراح إصلاحها بالذكاء الاصطناعي، مع بقاء الفحص الستاتيكي يعمل بدونه.
- تعلّم قواعد جديدة بمصادقة بشرية، وتعلّم من مراجعات المستخدم.

- تمييز المؤكَّد من المحتمل، ووضع صارم يعرض المؤكَّد فقط.
- تقييم أمني: خطورة لكل نتيجة، ودرجة وتقدير للمشروع، وربط بتصنيفي CWE و OWASP Top 10.
- واجهة كاملة وتقارير بصيغ قياسية.

---

3. ما يغطيه النظام

المتطلَّب	كيف نُفِّذ	أين في المستودع	الحالة
فحص محلي لمجلد يختاره المستخدم	قراءة الملفات وتحديد لغتها وتجاوز المجلدات غير المهمة	`sca/walker.py`	منفَّذ ومختبَر
تحليل ستاتيكي بقواعد	محرك قواعد بتعابير نمطية مع استثناءات	`sca/engines/rules_engine.py` و `sca/rules/`	منفَّذ ومختبَر
تحليل تدفق البيانات	شجرة AST لبايثون وتتبّع التلوّث بين الدوال (بعمق استدعاء واحد داخل الملف)	`sca/engines/python_ast.py`	منفَّذ ومختبَر
تتبّع مدخلات المستخدم للغات أخرى	تتبّع مبسّط للإسنادات في 9 لغات	`sca/engines/taint_lite.py`	منفَّذ ومختبَر، وهو تقريب
كشف الأسرار	صيغ مفاتيح معروفة + إنتروبيا شانون + حجب الأسرار في التقارير	`sca/engines/secrets.py`	منفَّذ ومختبَر
قواعد من مصادر مفتوحة	استيراد 300 قاعدة من DevSkim و Gitleaks مع فحص وقياس	`tools/import_rules.py` و `sca/rules/imported/`	منفَّذ ومقاس (القسم 5)
التحكم الكامل بالقواعد	عرض وتعديل وتعطيل واسترجاع وإنشاء وتجربة وتصدير واستيراد	`sca/rulebook.py` وتبويب القواعد	منفَّذ ومختبَر
أدوات خارجية	Bandit، واختيارياً Opengrep/Semgrep و Gitleaks	`sca/engines/external.py`	Bandit مختبَر، والبقية على مخرجات نموذجية
دمج النتائج	نتيجة واحدة لكل مشكلة، والثقة بقاعدة Noisy-OR	`sca/merge.py`	منفَّذ ومختبَر
مصنّف تعلّم آلة	TF-IDF مع انحدار لوجستي، يُعاد تدريبه بمراجعات المستخدم	`sca/ai/classifier.py`	منفَّذ ومختبَر، مدرَّب على بيانات اصطناعية
مراجعة بالذكاء الاصطناعي	تحقق وشرح وإصلاح، وفحص عميق لملفات كاملة	`sca/ai/llm.py`	منفَّذ، ومختبَر بخوادم وهمية فقط
مزوّدو الذكاء الاصطناعي	OpenAI و Gemini و Anthropic و OpenRouter و Groq و DeepSeek و Mistral وأي عنوان متوافق، أو Ollama محلياً	`sca/ai/providers.py`	منفَّذ، **ولم يُجرَّب على خدمة حقيقية**

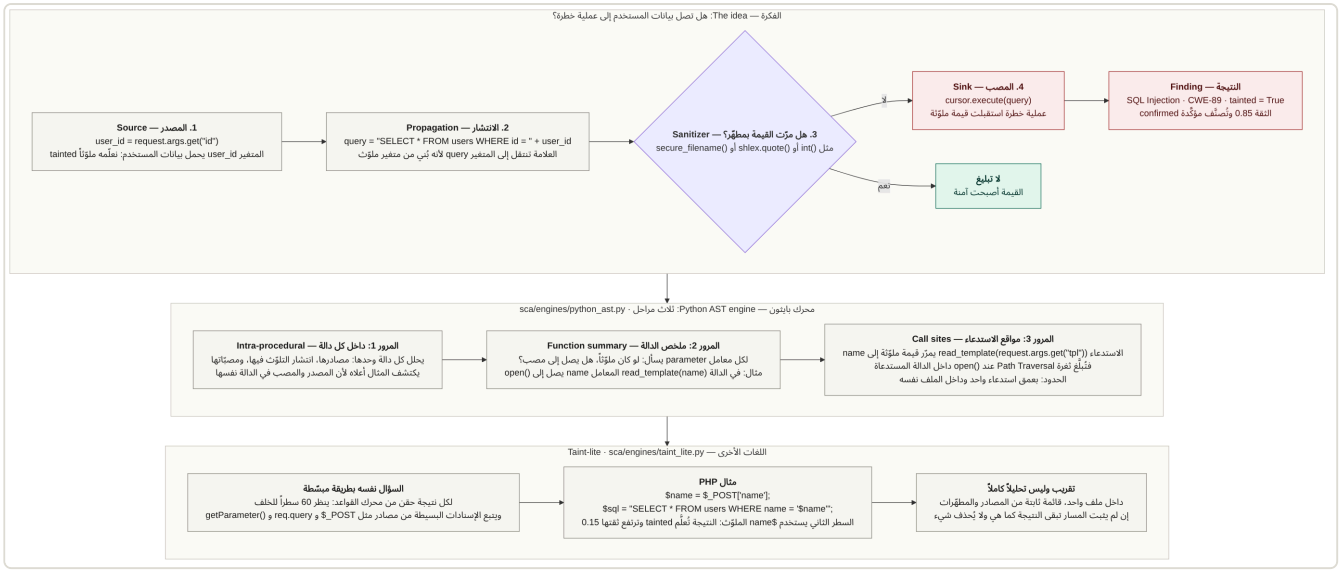
المتطلَّب	كيف نُفِّذ	أين في المستودع	الحالة
التعلّم بمصادقة	اقتراح، 9 فحوص آلية، تصحيح ذاتي مرة واحدة، مصادقة بشرية	`sca/learn/`	منفَّذ ومختبَر بنموذج وهمي
التعلّم من المراجعات	ثقة القاعدة تتبع سجلّها، وحجر القاعدة المتكررة الخطأ	`sca/learn/feedback.py`	منفَّذ ومختبَر
مؤكَّد ومحتمل ووضع صارم	دليل مستقل شرط للتأكيد	`sca/scoring.py`	منفَّذ ومقاس
التقييم الأمني	خطورة 0-10، درجة 0-100، تقدير A-F، CWE و OWASP Top 10	`sca/scoring.py` و `sca/cwe.py`	منفَّذ. الدرجة مقياس خاص بالأداة
الواجهة	لوحة، نتائج، قواعد، تعلّم، تقارير، سجل، تعليمات، بلغتين	`app.py` و `sca/ui/`	منفَّذ ومختبَر آلياً وفي متصفح
سطر الأوامر	scan و config و learn و rules و feedback و coverage و doctor و history	`sca/__main__.py`	منفَّذ ومختبَر
التقارير	HTML و PDF و JSON و SARIF 2.1.0	`sca/report/`	منفَّذ ومختبَر
السجل	قاعدة SQLite محلية وتغيّر الدرجة عبر الزمن	`sca/storage.py`	منفَّذ ومختبَر
التوثيق	دليل عربي، دليل القواعد، شرح المخططات باللغتين، المعمارية، هيكل البحث	`docs/` وتبويب التعليمات	منفَّذ

4. كيف يعمل: المراحل والتقنيات

#	المرحلة	التقنية
1	جمع الملفات	`os.walk` ، خريطة امتدادات، تجاوز الملفات الثنائية والمصغَّرة
2	الكشف	تعابير نمطية بقواعد YAML، مكتبة `ast` ، إنتروبيا شانون، Bandit
3	التنقية	تتبّع المدخلات، وتعديل الثقة بتوزيع بيتا: `(tp + 4c) / (tp + fp + 4)`
4	الدمج	الملف نفسه + نوع الثغرة نفسه + سطر متقارب، والثقة `1 - ∏(1 - c)`
5	المصنّف	TF-IDF لمقاطع من 1 إلى 3 رموز + انحدار لوجستي، بوزن ربع الثقة
6	النموذج اللغوي	طلبات JSON فقط، مرور ثانٍ للتأكد من الاكتشافات، دفاع ضد حقن التعليمات
7	التقييم	`risk = base × (0.6 + 0.4 × confidence)` و `score = 100 × exp(-(penalty / KLOC) / 25)`
8	التخزين والتقارير	SQLite، HTML مستقل، SARIF، fpdf2
9	التعلّم	قواعد يكتبها النموذج، تُجرَّب في عملية معزولة قابلة للإيقاف، ثم مصادقة بشرية

المخطط التفصيلي لكل مرحلة مع أدواتها في `docs/images/pipeline.ar.png` ، وشرحه في `docs/DIAGRAMS_AR.md` ، وهو معروض أيضاً داخل الواجهة.

تتبّع التلوّث هو أعمق ما في التحليل الستاتيكي هنا. الفكرة: القيمة القادمة من المستخدم تُعلَّم بأنها ملوّثة، وتنتقل العلامة عبر الإسناد وبناء النصوص، ويزيلها المعقِّم، فإن وصلت إلى عملية خطرة فهذه ثغرة بدليل.



---

5. نظام القواعد

5.1 المصادر

المصدر	العدد	الفعّال	الملف
مدمجة، كُتبت واختُبرت مع الأداة	98	98	`sca/rules/*.yaml`
مستوردة من Microsoft DevSkim (رخصة MIT)	95	49	`sca/rules/imported/devskim.yaml`
مستوردة من Gitleaks (رخصة MIT)، صيغ مفاتيح	205	204	`sca/rules/imported/gitleaks.yaml`
متعلَّمة من الذكاء الاصطناعي بعد المصادقة	تزداد مع الاستخدام		`learned/rules.yaml`
قواعد المستخدم	حسب ما يكتب		`learned/custom.yaml`

5.2 كيف استُوردت القواعد

لم تُنسخ القواعد المفتوحة المصدر كما هي. سكربت الاستيراد يحوّل كل قاعدة إلى صيغة المحرك ثم يختبرها:

1. يُترجَم التعبير في بايثون (الأصل مكتوب لـ .NET أو Go).
2. ينتهي على أسطر إجهاد طويلة ومتكررة، داخل عملية معزولة بحدّ زمني.
3. ينجح في أمثلة القاعدة نفسها: يطابق ما يجب ولا يطابق ما لا يجب.
4. لا يطابق شيئاً في الكود المعروف أنه آمن.
5. يُقاس على **2,384,751 سطر كود من 21 مشروعاً مفتوح المصدر معروفاً** (Django و Flask و Redis و Laravel و Express و Gson وغيرها).

المصدر	في المصدر	مفعّلة	معطّلة مع السبب	مستبعدة مع السبب
DevSkim	156	49	46	61

المصدر	في المصدر	مفعّلة	معطّلة مع السبب	مستبعدة مع السبب
Gitleaks	222	204	1	17

القاعدة تأتي معطَّلة إذا كان مصدرها يصنّفها «للمراجعة اليدوية»، أو طابقت كثيراً في الكود الحقيقي، أو قُرئت مطابقاتها يدوياً وتبيّن أنها غير مفيدة (ست قواعد، وسبب كل واحدة مكتوب معها). السجل الكامل في `evaluation/import_report.json`.

أثر الاستيراد المقاس على ثلاثة مشاريع كبيرة (1.17 مليون سطر):

المشروع	أسطر الكود	النتائج قبل ← بعد	المؤكَّدة قبل ← بعد	الزمن قبل ← بعد
redis	346,353	216 ← 199	2 ← 2	22 ث ← 15 ث
laravel/framework	373,034	398 ← 394	0 ← 0	22 ث ← 15 ث
django	449,759	496 ← 496	5 ← 5	36 ث ← 27 ث

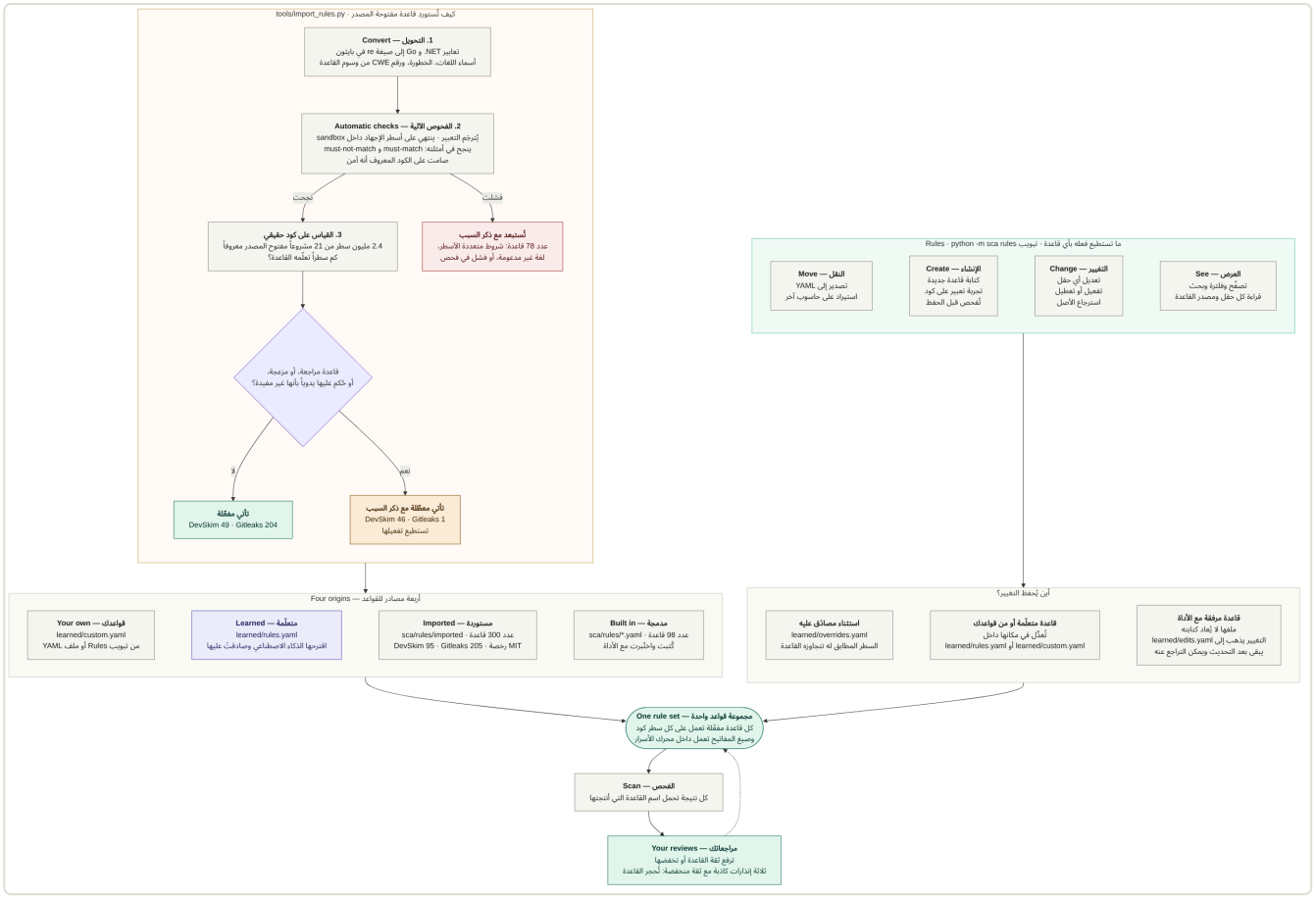
أي أن الاستيراد أضاف 21 نتيجة محتملة في 1.17 مليون سطر، ولم يضف نتائج مؤكَّدة، وجعل الفحص أبطأ بنحو 40%.

5.3 التحكم بالقواعد

من تبويب **القواعد** في الواجهة، أو بالأمر `python -m sca rules`:

- **العرض:** كل القواعد مع فلاتر (المصدر، اللغة، الثغرة، الحالة) وبحث، وكل حقول القاعدة ومصدرها ورخصتها.
- **التعديل:** أي حقل (النمط، الاستثناء، الخطورة، الثقة، اللغات، الشرح)، والتفعيل والتعطيل، واسترجاع الأصل.
- **الإنشاء:** نموذج لكتابة قاعدة جديدة تُفحص قبل الحفظ، وصفحة لتجربة تعبير على كود.
- **النقل:** تصدير واستيراد بصيغة YAML.

القواعد المرفقة مع الأداة لا يُعاد كتابة ملفاتها: تعديل المستخدم يُحفظ في ملف منفصل (`learned/edits.yaml`)، فيبقى بعد التحديث ويمكن التراجع عنه. والقاعدة الجديدة تُرفض إذا كان فيها حقل خاطئ، أو تعبير لا يُترجَم أو بطيء جداً، أو فشلت في مثال من أمثلتها.



Deploy

Secure Code Analyzer

اللغة / Language

English العربية

كشف ثغرات ستاتيكي وبالذكاء الاصطناعي
مع تعلّم مستمر - الإصدار 1.1.0

1. مجلد المشروع

مسار المجلد

/home/claude/graduate_project/sam

استعراض... استخدم العيّنة

2. المحركات الستاتيكية

قواعد الأنماط

352 قاعدة فعّالة من 399 - تُدار من تبويب القواعد

تحليل بايثون (AST وتتبّع التلوّث)

كشف الأسرار

Bandit

Opengrep / Semgrep (غير منصّب)

Gitleaks (غير منصّب)

خيارات إضافية للمحركات

3. الذكاء الاصطناعي

مصنّف تعلّم الآلة (رأي ثانٍ)

اللوحة النتائج القواعد التعلّم التقارير السجل التعليمات

قواعدك	متعلَّمة	مستوردة	مدمجة	الفعّالة	كل القواعد
1	0	300	98	352	399

الفعّالة / الكل: مدمجة 98/98 · مستوردة 253/300 · متعلَّمة 0/0 · قواعدك 1/1 · عدّلتَها 0

كل القواعد التي تعرفها الأداة. تصفّحها وفلترها، واقرأ أي حقل وغيّره، وعطّل قاعدة أو فعّلها، واكتب قاعدتك، وجرّب تعبيراً على كود. لا شيء هنا يحتاج الذكاء الاصطناعي.

تصفّح وتعديل قاعدة جديدة تجربة تعبير استيراد وتصدير

بحث (الاسم، العنوان، التعبير) | الحالة: الكل | الثغرة: الكل | اللغة: الكل | المصدر: الكل

399 من 399 قاعدة.

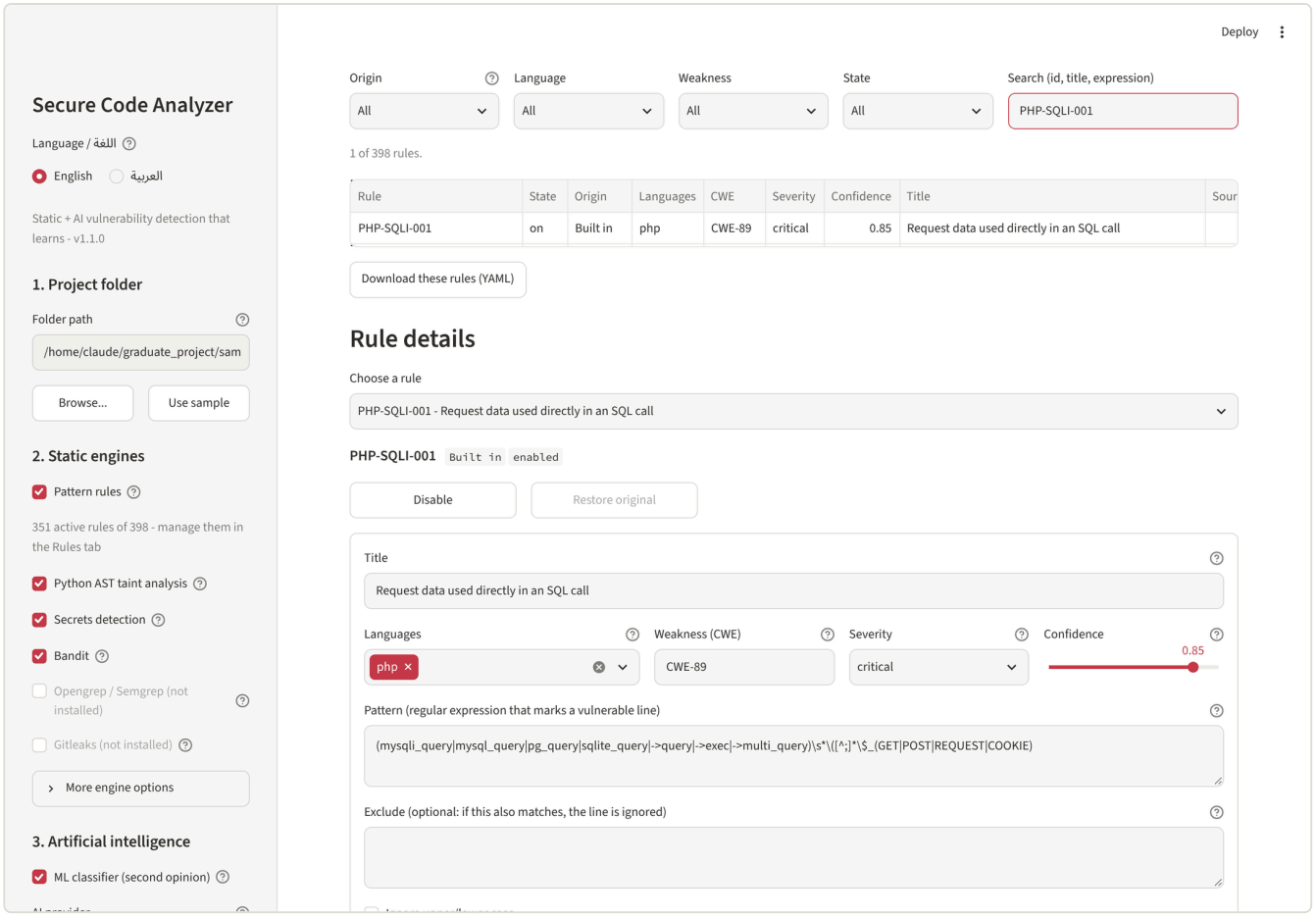
الأصل	العنوان	الثقة	الخطورة	CWE	اللغات	المصدر	الحالة	القاعدة
	gets() can never be used safely	0.95	critical	CWE-242	c	مدمجة	تعمل	C-BOF-001
	Unbounded string copy (strcpy / strcat)	0.6	high	CWE-120	c	مدمجة	تعمل	C-BOF-002
	Unbounded formatted write (sprintf)	0.6	high	CWE-120	c	مدمجة	تعمل	C-BOF-003
	scanf family with an unbounded %s	0.8	high	CWE-120	c	مدمجة	تعمل	C-BOF-004
	Non-constant format string	0.75	high	CWE-134	c	مدمجة	تعمل	C-FMT-001
	system()/popen() called with a non-constant command	0.7	critical	CWE-78	c	مدمجة	تعمل	C-CMDI-001
	rand() is not suitable for security values	0.45	low	CWE-338	c	مدمجة	تعمل	C-RANDOM-001
	Predictable temporary file name	0.8	medium	CWE-377	c			C-TMP-001

نزّل هذه القواعد (YAML)

تفاصيل القاعدة

اختر قاعدة

MY-NODE-EXEC-001 - child_process.exec built from a template string



6. الذكاء الاصطناعي وحلقة التعلّم

ثلاثة أدوار للذكاء الاصطناعي:

1. **مصنّف محلي** يعطي كل نتيجة درجة، ويُعاد تدريبه بالأسطر التي يراجعها المستخدم.
2. **مراجعة النتائج:** النموذج اللغوي يقرأ النتيجة مع الكود المحيط، ويحكم ويشرح ويقترح إصلاحاً. النتيجة التي يحكم بأنها كاذبة تبقى وتُعلَّم ولا تُحذف.
3. **الفحص العميق:** النموذج يدقق ملفات كاملة ويبلّغ عمّا لم تجده القواعد، وطلب ثانٍ مستقل يجب أن يؤكد كل اكتشاف.

حلقة التعلّم:

1. **الاقتراح:** لكل اكتشاف مؤكَّد يكتب النموذج قاعدة بصيغة JSON مع أمثلة. ولكل إنذار كاذب يكتب استثناءً ضيقاً.
2. **تسعة فحوص آلية:** البنية صحيحة، التعبير يُترجَم، سريع على أسطر الإجهاد، محدَّد وليس عاماً، يطابق السطر الذي تعلّم منه، ينجح في أمثلته، صامت على الكود الآمن، لا يبلّغ عن نسبة غير معقولة من المشروع، وليس مكرَّراً.
3. **تصحيح ذاتي:** عند الفشل تُعاد أسباب الفشل إلى النموذج مرة واحدة.
4. **المصادقة البشرية:** القاعدة الناجحة تنتظر في تبويب التعلّم مع نتائج فحوصها.
5. **فترة تجربة:** القاعدة المصادَق عليها تبدأ بثقة 0.55، ثم ترفعها مراجعات المستخدم أو تحجرها.

يمكن تكرار الحلقة حتى لا تجد جولة شيئاً جديداً (`--rounds`)، وطلب قواعد لكل ثغرة لا قاعدة لها في أي لغة (`bootstrap all`). المخطط التفصيلي في `docs/images/learning-loop.ar.png` .

الخصوصية: مع مزوّد سحابي تُرسَل أجزاء من الكود إلى المزوّد، وتُحجب قبل ذلك الأسرار التي اكتشفتها الأداة فقط. مع Ollama لا يخرج شيء من الحاسوب. مفتاح الـ API يُحفظ في ملف `.env` مستثنى من git ولا يُكتب في التقارير ولا في قاعدة البيانات.

---

7. تقليل الإنذارات الكاذبة

لا يستطيع أي محلل ستاتيكي أن يضمن صفر إنذارات كاذبة، فالمسألة غير قابلة للحسم نظرياً. ما يفعله النظام هو طبقات متتالية:

الطبقة	الأثر
استثناءات داخل القواعد وقوائم معقِّمات	الصيغ الآمنة لا يُبلَّغ عنها
تتبّع المدخلات	يفرّق بين «بيانات المستخدم تصل إلى السطر» و«طابق نمط»
عدة محركات مع Noisy-OR	الاتفاق يرفع الثقة
مراجعة الذكاء الاصطناعي بمرور ثانٍ	الكاذب يُعلَّم، والاكتشاف يحتاج تأكيداً
مراجعات المستخدم	المرفوض يبقى مرفوضاً، والقاعدة المزعجة تفقد ثقتها ثم تُحجر
كود الاختبارات	ثقة أقل، ولا تأكيد تلقائي
الوضع الصارم	يُحتسب فقط ما له دليل مستقل، والباقي يُنقل إلى «تحتاج مراجعة»

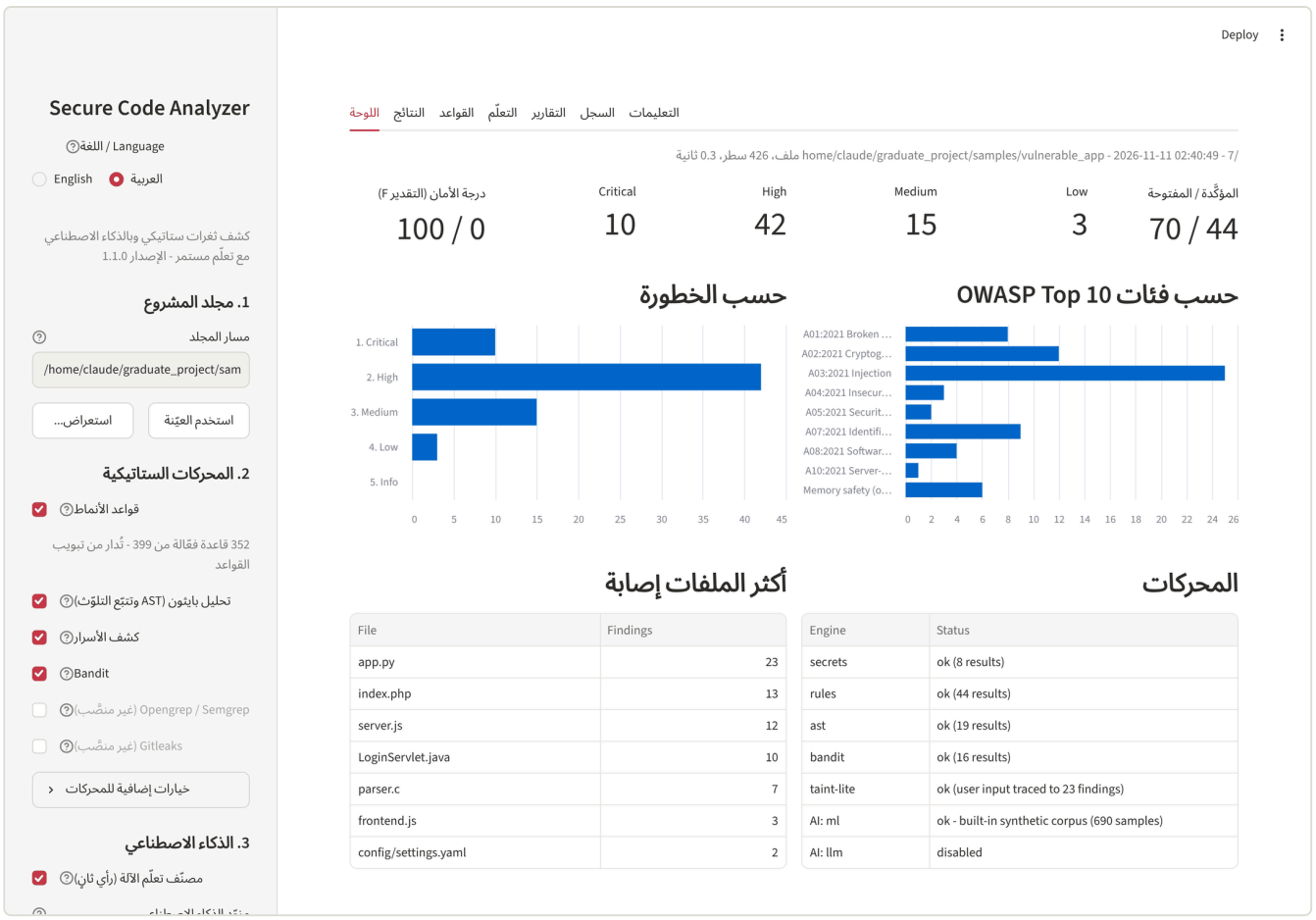
ثغرات الحقن (SQL، أوامر النظام، XSS، المسارات، eval، إلغاء التسلسل) لا تُؤكَّد باتفاق محركين ولا بثقة عالية، لأنها تحتاج إثبات تدفق بيانات. يؤكّدها تتبّع المدخلات أو المراجعة فقط. هذا القرار جاء بعد تجربة على كود حقيقي: القاعدة الأولى اعتبرت 56 نتيجة في مكتبة pandas مؤكَّدة، أغلبها في ملفات الاختبار، وبعد التعديل صار العدد صفراً.

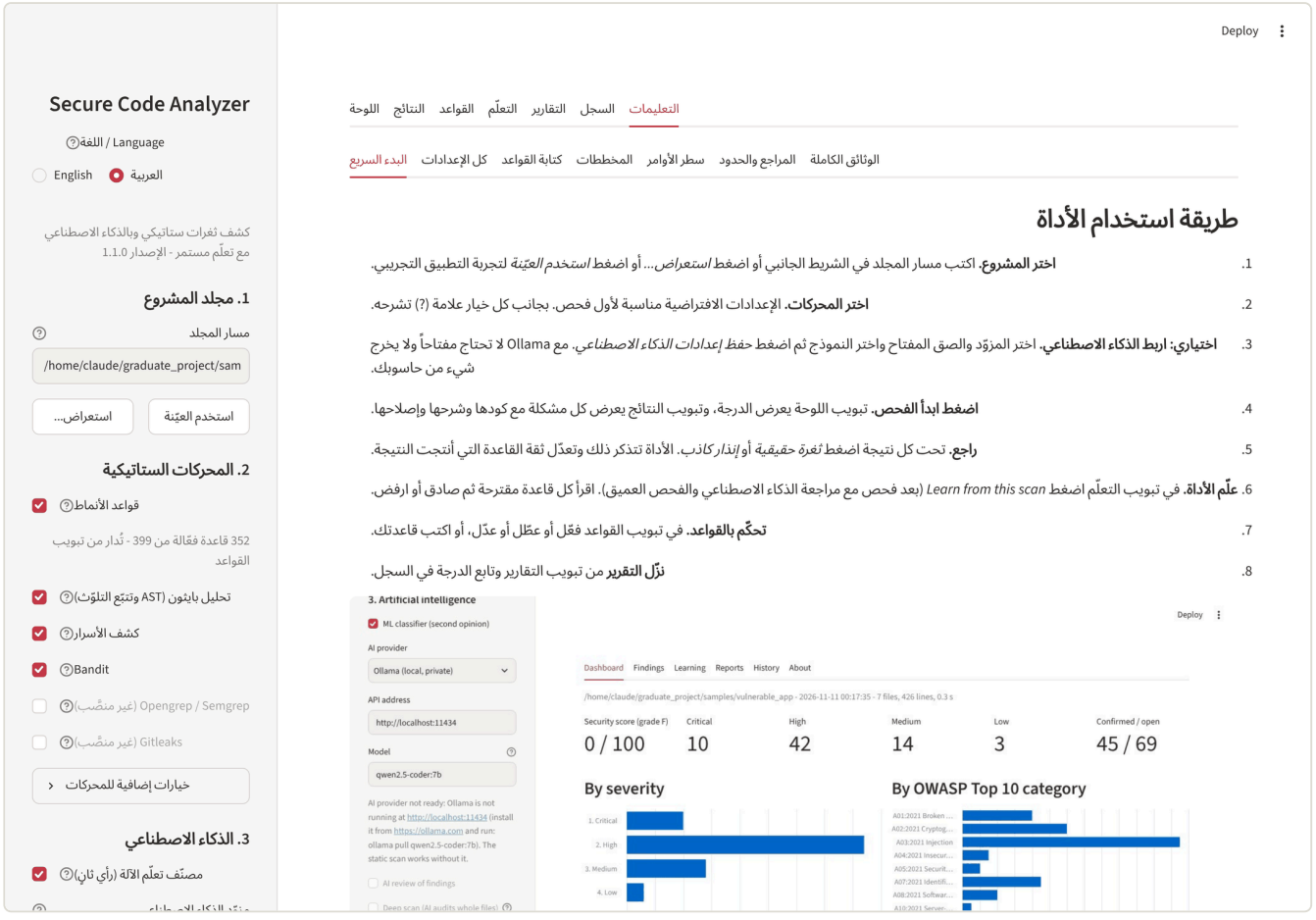
---

8. الواجهة

سبعة تبويبات، بالعربية أو الإنجليزية، وبجانب كل خيار شرح لوظيفته:

التبويب	المحتوى
اللوحة	الدرجة والتقدير، الأعداد حسب الخطورة وفئات OWASP، أكثر الملفات إصابة، المحركات
النتائج	كل نتيجة مع كودها وسبب تأكيدها وشرحها وإصلاحها، وزرّا المراجعة
القواعد	التحكم الكامل بالقواعد (القسم 5.3)
التعلّم	القواعد المقترحة وفحوصها والمصادقة، سجل أداء القواعد، التغطية لكل لغة
التقارير	HTML و PDF و JSON و SARIF
السجل	الفحوص السابقة وتغيّر الدرجة
التعليمات	البدء السريع، وظيفة كل إعداد، كتابة القواعد، المخططات السبعة مع شرحها باللغتين، سطر الأوامر، المراجع، والوثائق الكاملة


Deploy
Secure Code Analyzer
اللغة / Language
English
العربية
كشف ثغرات ستاتيكي وبالذكاء الاصطناعي مع تعلّم مستمر - الإصدار 1.1.0
1. مجلد المشروع
مسار المجلد
/home/claude/graduate_project/sam
استعراض...
استخدم العيّنة
2. المحركات الستاتيكية
قواعد الأنماط
352 قاعدة فعّالة من 399 - تُدار من تبويب القواعد
تحليل بايثون (AST وتتبّع التلوّث)
كشف الأسرار
Bandit
Opengrep / Semgrep (غير منصّب)
Gitleaks (غير منصّب)
خيارات إضافية للمحركات
3. الذكاء الاصطناعي
مصنّف تعلّم الآلة (رأي ثانٍ)
اللوحة النتائج القواعد التعلّم التقارير السجل التعليمات
/home/claude/graduate_project/samples/vulnerable_app - 2026-11-11 02:40:49 - 7/ ملف، 426 سطر، 0.3 ثانية
درجة الأمان (التقدير F)
100 / 0
Critical
10
High
42
Medium
15
Low
3
المؤكَّدة / المفتوحة
70 / 44
حسب الخطورة
1. Critical
2. High
3. Medium
4. Low
5. Info
0 5 10 15 20 25 30 35 40 45
حسب فئات OWASP Top 10
A01:2021 Broken ...
A02:2021 Cryptog...
A03:2021 Injection
A04:2021 Insecur...
A05:2021 Securit...
A07:2021 Identifi...
A08:2021 Softwar...
A10:2021 Server-...
Memory safety (o...
0 2 4 6 8 10 12 14 16 18 20 22 24 26
أكثر الملفات إصابة
File
Findings
app.py 23
index.php 13
server.js 12
LoginServlet.java 10
parser.c 7
frontend.js 3
config/settings.yaml 2
المحركات
Engine
Status
secrets ok (8 results)
rules ok (44 results)
ast ok (19 results)
bandit ok (16 results)
taint-lite ok (user input traced to 23 findings)
AI: ml ok - built-in synthetic corpus (690 samples)
AI: llm disabled


Deploy
Secure Code Analyzer
اللغة / Language
English
العربية
كشف ثغرات ستاتيكي وبالذكاء الاصطناعي مع تعلّم مستمر - الإصدار 1.1.0
1. مجلد المشروع
مسار المجلد
/home/claude/graduate_project/sam
استعراض...
استخدم العيّنة
2. المحركات الستاتيكية
قواعد الأنماط
352 قاعدة فعّالة من 399 - تُدار من تبويب القواعد
تحليل بايثون (AST وتتبّع التلوّث)
كشف الأسرار
Bandit
Opengrep / Semgrep (غير منصّب)
Gitleaks (غير منصّب)
خيارات إضافية للمحركات
3. الذكاء الاصطناعي
مصنّف تعلّم الآلة (رأي ثانٍ)
اللوحة النتائج القواعد التعلّم التقارير السجل التعليمات
الوثائق الكاملة المراجع والحدود سطر الأوامر المخططات كتابة القواعد كل الإعدادات البدء السريع
طريقة استخدام الأداة
1. اختر المشروع. اكتب مسار المجلد في الشريط الجانبي أو اضغط استعراض... أو اضغط استخدم العيّنة لتجربة التطبيق التجريبي.
2. اختر المحركات. الإعدادات الافتراضية مناسبة لأول فحص. بجانب كل خيار علامة (?) تشرحه.
3. اختياري: اربط الذكاء الاصطناعي. اختر المزوّد والصق المفتاح واختر النموذج ثم اضغط حفظ إعدادات الذكاء الاصطناعي. مع Ollama لا تحتاج مفتاحاً ولا يخرج شيء من حاسوبك.
4. اضغط ابدأ الفحص. تبويب اللوحة يعرض الدرجة، وتبويب النتائج يعرض كل مشكلة مع كودها وشرحها وإصلاحها.
5. راجع. تحت كل نتيجة اضغط ثغرة حقيقية أو إنذار كاذب. الأداة تتذكر ذلك وتعدّل ثقة القاعدة التي أنتجت النتيجة.
6. علّم الأداة. في تبويب التعلّم اضغط Learn from this scan (بعد فحص مع مراجعة الذكاء الاصطناعي والفحص العميق). اقرأ كل قاعدة مقترحة ثم صادق أو ارفض.
7. تحكّم بالقواعد. في تبويب القواعد فعّل أو عطّل أو عدّل، أو اكتب قاعدتك.
8. نزّل التقرير من تبويب التقارير وتابع الدرجة في السجل.
3. Artificial intelligence
ML classifier (second opinion)
AI provider
Ollama (local, private)
API address
http://localhost:11434
Model
qwen2.5-coder:7b
AI provider not ready: Ollama is not running at http://localhost:11434 (install it from https://ollama.com and run: ollama pull qwen2.5-coder:7b). The static scan works without it.
AI review of findings
Deep scan (AI audits whole files)
Dashboard Findings Learning Reports History About
/home/claude/graduate_project/samples/vulnerable_app - 2026-11-11 00:17:35 - 7 files, 426 lines, 0.3 s
Security score (grade F)
0 / 100
Critical
10
High
42
Medium
14
Low
3
Confirmed / open
45 / 69
By severity
By OWASP Top 10 category

9. النتائج المقاسة

9.1 على العيّنات الموسومة

العيّنة: تطبيق مصاب عمداً فيه 71 ثغرة موسومة (5 منها صعبة عمداً) و 21 سطراً آمناً يشبه المصاب، ومعه تطبيق آمن كل نتيجة فيه إنذار كاذب. الأمر: `python evaluation/evaluate.py` .

الإعداد	TP	FP	FN	الدقة	الاسترجاع	F1
Bandit فقط (خط الأساس لبايثون)	15	4	56	0.789	0.211	0.333
قواعد الأنماط + الأسرار	48	3	23	0.941	0.676	0.787
+ محرك بايثون AST	66	3	5	0.957	0.930	0.943
كل المحركات الستاتيكية	66	7	5	0.904	0.930	0.917
كل المحركات، الوضع الصارم	43	0	28	1.000	0.606	0.754

9.2 على كود حقيقي (اختبار ضجيج، وليس قياس دقة)

أربع مكتبات معروفة (قبل استيراد القواعد): pandas و scikit-learn و streamlit و bandit، مجموعها 1.19 مليون سطر. النتيجة: 195 نتيجة، 133 منها في كود الاختبارات، وواحدة فقط مؤكَّدة (استخدام MD5 لبصمة ملف بيانات). وأثر استيراد القواعد على ثلاثة مشاريع أخرى في القسم 5.2.

9.3 كيف تُقرأ هذه الأرقام

- العيّنات كُتبت مع الأداة، وبعض القواعد ضُبطت أثناء النظر إليها. فهي تثبت أن السلسلة تعمل، ولا تثبت الأداء على كود لم تره الأداة.
- الوضع الصارم وصل إلى صفر إنذارات كاذبة **على هذه العيّنات** بثمن: ثلث الثغرات الحقيقية نُقل إلى «تحتاج مراجعة».
- الكود الحقيقي غير موسوم، فلا تُحسب منه دقة ولا استرجاع. ما يُستفاد منه أن الأداة هادئة على كود ناضج.

10. الاختبار والتحقق

ما اختُبر	كيف
كل محرك بحالات مصابة وحالات آمنة	اختبارات آلية
السلسلة كاملة، التقارير، قاعدة البيانات	اختبارات آلية
عملاء المزوّدين وحلقة التعلّم	خوادم محلية وهمية تقلّد صيغ Ollama و OpenAI و Anthropic، ونموذج وهمي إجاباته مكتوبة في الاختبار
القواعد المستوردة	كل قاعدة تُترجَم، وتنجح في أمثلتها، ولها مصدر ورخصة، والمعطّلة لها سبب

ما اختُبر	كيف
التحكم بالقواعد	إنشاء، رفض القاعدة الخاطئة بسبب واضح، تعديل، تعطيل، استرجاع، حذف، تصدير، استيراد
الواجهة	اختبارات آلية لكل المسارات، ثم تشغيل في متصفح حقيقي عبر كل التبويبات باللغتين
أنظمة التشغيل	لينكس وويندوز، بايثون 3.10 و 3.12 (GitHub Actions)، ومحلياً 3.13

المجموع 271 اختباراً آلياً. التشغيل: `pip install -r requirements-dev.txt` ثم `python -m pytest tests -q` .

---

11. الحدود وما لم يُختبر

هذه النقاط يجب أن يعرفها المشرف قبل المناقشة:

1. **لم يُجرَ أي اتصال بخدمة ذكاء اصطناعي حقيقية.** عملاء المزوّدين وحلقة التعلّم مختبَرة بخوادم وهمية فقط، لأن مفتاح API لم يكن متاحاً أثناء التطوير. جودة المراجعة وجودة القواعد التي يكتبها نموذج حقيقي غير مقاسة.
2. **صفر إنذارات كاذبة غير مضمون.** الوضع الصارم يقلّلها بثمن في الاسترجاع.
3. **العيّنات ليست محايدة**، فهي مكتوبة مع الأداة.
4. **القواعد المستوردة** مفحوصة آلياً ومقاسة على كود حقيقي، لكنها لم تُثبَت واحدةً واحدة على ثغرات حقيقية ولا على مفاتيح مسرَّبة حقيقية.
5. **القاعدة تعبير نمطي على سطر واحد**، فلا تصف ثغرة تمتد على عدة أسطر أو تعتمد على منطق البرنامج.
6. **تحليل بايثون بين الدوال** بعمق استدعاء واحد وداخل الملف نفسه. واللغات الأخرى تُفحص سطراً سطراً مع تتبّع مبسّط.
7. **Dart و Perl و Lua و Elixir** بلا قواعد فعّالة خاصة بها، ويغطيها الفحص العميق والقواعد العامة وكشف الأسرار.
8. **المصنّف مدرَّب على بيانات اصطناعية**، ووزنه محدود لهذا السبب.
9. **Opengrep/Semgrep و Gitleaks الخارجيان** مختبَران على مخرجات نموذجية فقط.
10. **درجة الأمان** مقياس خاص بالأداة وليست درجة CVSS.

---

12. ما يتبقى على الطالب

1. تشغيل المنصة بمفتاح API حقيقي، وقياس: دقة مراجعة النموذج على العيّنات، وعدد القواعد التي يقترحها، ونسبة ما ينجح منها في الفحوص التسعة، وأثرها على الاسترجاع في الفحص التالي.
2. تقييم على بيانات خارجية لم تُكتب مع الأداة (مثل مجموعة OWASP Benchmark أو Juliet).
3. مقارنة مع أداة أخرى على المشروع نفسه (مثل Semgrep أو CodeQL).
4. كتابة البحث نفسه. الهيكل المقترح مع الجداول الجاهزة في `docs/THESIS_OUTLINE_AR.md` .

---

13. طريقة التشغيل للمشرف

```
git clone https://github.com/blackangiliq/graduate_project.git
cd graduate_project
./run.sh            # انقر مرتين على ويندوز: run.bat على
```

يفتح المتصفح على `http://localhost:8501`. اختر **العربية** من أعلى الشريط الجانبي، ثم اضغط **استخدم العيّنة** ثم **ابدأ الفحص**. بعدها:

- تبويب **النتائج** لرؤية الثغرات وشرحها.
- تبويب **القواعد** لتصفّح القواعد وتعديل واحدة أو كتابة جديدة.
- تبويب **التعليمات** لكل الشروحات والمخططات.

ومن سطر الأوامر:

```
python -m sca scan samples/vulnerable_app
python -m sca rules catalog --lang php
python -m sca doctor
python evaluation/evaluate.py
```

14. أين يجد المشرف كل شيء

الوثيقة	المحتوى
`README.md`	نظرة عامة بالإنجليزية مع صور الواجهة
`docs/GUIDE_AR.md`	الدليل الكامل بالعربية: التشغيل، العمل الداخلي، التعلّم، القواعد، أسئلة المناقشة
`docs/RULES.md`	دليل القواعد بالعربية والإنجليزية مع جداول الاستيراد
`docs/DIAGRAMS_AR.md`	شرح المخططات السبعة بالعربية مع مثال متتبَّع بالأرقام
`docs/DIAGRAMS.md`	شرح المخططات بالإنجليزية
`docs/ARCHITECTURE.md`	المعمارية التقنية بالإنجليزية
`docs/THESIS_OUTLINE_AR.md`	هيكل بحث التخرج فصلاً فصلاً
`evaluation/results.md`	نتائج التقييم على العيّنات
`evaluation/import_report.json`	سجل استيراد القواعد: كل قاعدة وقرارها وقياسها
`THIRD_PARTY_NOTICES.md`	رخص المصادر المستوردة

15. المراجع الأساسية

- OWASP Top 10 (2021): https://owasp.org/Top10/
- MITRE CWE: https://cwe.mitre.org/
- SARIF 2.1.0 (OASIS): https://www.oasis-open.org/committees/sarif
- Microsoft DevSkim: https://github.com/microsoft/DevSkim
- Gitleaks: https://github.com/gitleaks/gitleaks
- Bandit: https://github.com/PyCQA/bandit
- Opengrep: https://github.com/opengrep/opengrep
- Livshits and Lam, *Finding Security Vulnerabilities in Java Applications with Static Analysis*, USENIX Security 2005: https://www.usenix.org/conference/14th-usenix-security-symposium/finding-security-vulnerabilities-java-applications-static
- Feng et al., *CodeBERT: A Pre-Trained Model for Programming and Natural Languages*, 2020: https://arxiv.org/abs/2002.08155

القائمة الكاملة مع الروابط في تبويب **التعليمات** ← **المراجع والحدود**.